BÁO CÁO TEST HIỆU NĂNG

Sản phẩm: ZeroCheck AI · https://cbyts.vnpt.vn

Phạm vi & trung thực về phương pháp: đội đo **độ trễ thực tế theo thao tác** (nhiều lần lặp, trên máy dev và trên production), kiểm chứng các **cơ chế chịu tải theo thiết kế**, và **đã chạy load test bằng k6** trên đường nóng nghiệp vụ - trên máy dev (mục 5) và **trên chính cụm K8s của VNPT Cloud** (mục 5b), kèm kết xuất thô của k6. Không suy diễn p95/p99 khi chưa đo: mọi con số dưới đây là **số đo thật**, kèm điều kiện đo.

1. Độ trễ thực đo theo thao tác

1.1. Môi trường dev (Windows 11, PostgreSQL local)

Thao tác	Độ trễ điển hình	Ghi chú
Kiểm tra 1 hồ sơ XML 130 (bộ máy luật, không AI)	**p95 173 ms** (trung vị 125 ms)	trọn gói: phân tích 15 bảng + 455 đặc tả + 6 nhóm luật - số k6 ở mục 5
Cảnh báo nhập liệu (luật cục bộ trong trình duyệt)	**< 50 ms**	không gọi mạng
Nạp văn bản 100 trang (~300 KB text)	**< 2 s**	chia đoạn + chỉ mục toàn văn tự sinh
Truy hồi ngữ cảnh (tìm đoạn liên quan)	**< 50 ms**	chỉ mục GIN + HNSW
Hỏi đáp AI trọn vòng	2–6 s	phần chậm là AI sinh câu trả lời
Kiểm tra + nhận định AI	2–8 s	chỉ khi bật AI

1.2. Trên production (VNPT Cloud, đo thật 28/07/2026)

Một lượt hỏi đáp RAG đầy đủ, bắt từ log pod (chi tiết trong tài liệu *Minh chứng AI*):

Chặng	Thời gian
Vector hóa câu hỏi (Ollama bge-m3, trong cụm)	248 ms
Truy hồi + chọn lọc ngữ cảnh (pgvector + toàn văn)	~4 s (gồm AI chấm điểm rerank)
LLM sinh câu trả lời (OpenRouter gpt-4o-mini, 169 token)	~3 s
Tổng đầu–cuối phía người dùng	**9,49 s**

> Đường "nóng" của nghiệp vụ (cảnh báo nhập liệu, kiểm tra hồ sơ) **không đi qua AI** - luôn ở mức ms. AI là lớp tùy chọn, chậm cũng không chặn nghiệp vụ.

2. Thiết kế cho hiệu năng (đã kiểm chứng hoạt động)

Cơ chế	Tác dụng
Đặc tả trường + ~15.000 mã ICD **đệm sẵn trong bộ nhớ**	tra tức thời, không truy vấn CSDL trong vòng lặp luật
Chỉ mục toàn văn GIN + vector HNSW **tính sẵn lúc ghi**	đọc nhanh, chi phí trả một lần khi nạp văn bản
Luật chạy **độc lập, an toàn khi lỗi**	một luật hỏng bị bỏ qua, độ trễ ổn định
API **không giữ trạng thái**	thêm replicas là tăng sức tải; PostgreSQL là điểm chung duy nhất
Vector hóa nền (worker) sau khi nạp văn bản	upload trả về ngay, không chờ embedding
Kiểm tra lô kiểu **streaming** (NDJSON)	lô hàng nghìn hồ sơ trả kết quả dần, không chờ một cục

3. Giới hạn bảo vệ & kiểm soát tải (đã test thủ công)

Giới hạn	Giá trị	Đã kiểm
Upload XML / văn bản / lô batch	50 MB / 20 MB / 1 GB	✅ vượt → từ chối đúng
Rate limit chung	100 yêu cầu/phút/IP	✅ vượt → 429 + Retry-After
Đăng nhập / quên mật khẩu / hỏi AI / nạp đặc tả	5 / 3 / 10 / 5 mỗi phút/IP	✅ (chỉnh runtime trên admin)
Từ chối ngay, không xếp hàng (QueueLimit=0)	—	✅ không giữ kết nối gây dồn ứ

4. Phát hiện hiệu năng/độ bền đáng chú ý (tìm ra và xử lý thật)

Phát hiện	Xử lý
Load balancer VNPT **cắt kết nối im lặng ~5 s**, trong khi 1 lượt RAG mất 15–18 s → client báo "không kết nối được"	Server stream **keepalive** (byte trắng mỗi 2 s, hợp lệ JSON) + tắt buffer 2 tầng nginx — client không phải sửa
Một văn bản lớn (1.852 đoạn) lấn át toàn bộ kết quả truy hồi	Truy hồi 2 bước: vét rộng rồi AI chấm điểm, mỗi văn bản ≤ 3 đoạn
Kiểm tra lô lớn tải một cục gây chờ lâu	Chuyển endpoint lô sang streaming từng hồ sơ

5. Load test k6 - đã chạy thật

Kịch bản nằm trong repo: `loadtest/xml130-validate.k6.js` (repo `hackathon-api-validation`). Đường đo là đường nóng nghiệp vụ `POST /api/xml130/validate?useAi=false` - bộ máy luật thuần, không gọi AI.

5.1. Kết quả

Chỉ số	1 người dùng (1 VU / 45 s)	20 người dùng đồng thời (20 VU / 60 s)
Trung vị	124,78 ms	1,24 s
p95	**172,94 ms**	**1,85 s**

Chỉ số	1 người dùng (1 VU / 45 s)	20 người dùng đồng thời (20 VU / 60 s)
p99	**200,38 ms**	**2,12 s**
Thông lượng	7,57 hồ sơ/giây	**15,57 hồ sơ/giây**
Số lượt	341	947
Tỉ lệ lỗi HTTP	**0%**	**0%**
Kiểm tra nội dung phản hồi	682/682 đạt	1894/1894 đạt

Độ trễ gần như không đổi theo cỡ hồ sơ - p95 ở 1 VU: 166,4 ms (7 KB) / 159,8 ms (57 KB) / 186,2 ms (108 KB). Chi phí nằm ở số lượng luật chạy trên mỗi hồ sơ, không ở khối lượng byte phải phân tích.

5.2. Đo trong điều kiện nào

Yếu tố	Giá trị lúc đo
Máy	1 máy để bàn Windows 11 chạy đồng thời cả API, k6 và PostgreSQL - **không** phải máy chủ riêng
Cách chạy API	bản **Release**, Kestrel `http://localhost:5075`
Dữ liệu	3 hồ sơ XML 130 **thật** (7 / 57 / 108 KB), xoay vòng
Chống nhiễu	mỗi request mang một `X-Forwarded-For` giả lập riêng, để phép đo phản ánh năng lực bộ máy luật chứ không đụng trần chống spam
Kết xuất thô	`loadtest/ket-qua-release-1vu.json`, `loadtest/ket-qua-release-20vu.json` (sinh bằng `k6 run --summary-export=...`), kèm một lần chạy đối chứng bản Debug lệch ~3%

Lệnh lặp lại:

```
k6 run -e BASE_URL=http://localhost:5075 -e VUS=1  -e DURATION=45s \
  --summary-export=loadtest/ket-qua-release-1vu.json  loadtest/xml130-validate.k6.js
k6 run -e BASE_URL=http://localhost:5075 -e VUS=20 -e DURATION=60s \
  --summary-export=loadtest/ket-qua-release-20vu.json loadtest/xml130-validate.k6.js
```

5.3. Hai lưu ý đọc số cho đúng

1. Đây là **cận dưới của năng lực thật trên phần cứng đó**: k6 tranh CPU với chính API trên cùng một máy dev.
2. k6 báo *"thresholds have been crossed"* ở lần 20 VU vì ngưỡng trong script (`p(95)<500`) viết cho tình huống **một người dùng**. Vượt ngưỡng ở 20 VU là đúng dự đoán, không phải lỗi - tỉ lệ lỗi HTTP vẫn 0%.

Riêng endpoint hỏi đáp AI bị rate-limit 10/phút/IP theo thiết kế (kiểm soát chi phí LLM) - muốn load test phần đó phải nâng ngưỡng tạm thời trên web admin; đây là tính năng, không phải giới hạn kỹ thuật.

5b. Load test trên cụm K8s thật

Theo yêu cầu của BTC, đội đã đo **trên chính cụm VNPT Cloud**. Cách làm: dựng namespace riêng `cbyts-loadtest` - cùng image, cùng `resources.limits`, có PostgreSQL riêng - chạy k6 **bên trong cụm** (không qua Internet, không lẫn độ trễ WAN), đo xong xóa cả namespace. Namespace `cbyts` đang phục vụ thật **không nhận một request nào** của phép đo này.

Lượt đo	Replica API	p95	p99	Trung vị	Thông lượng	Lỗi HTTP	Check nội dung
1 VU / 45 s	1	**251 ms**	345 ms	194 ms	5,00 hồ sơ/s	**0%**	450/450 đạt
20 VU / 60 s	1	6,99 s	9,38 s	5,49 s	3,63 hồ sơ/s	**0%**	456/456 đạt
20 VU / 60 s	**3**	3,87 s	5,19 s	**1,44 s**	**10,83 hồ sơ/s**	**0%**	1320/1320 đạt

Kết xuất thô của cả 3 lượt nằm trong repo: `loadtest/ket-qua-cum-1vu.json`, `ket-qua-cum-20vu.json`, `ket-qua-cum-20vu-3replica.json`. Manifest dựng lại môi trường đo: `k8s/loadtest/`.

Hai điều rút ra:

1. **Một pod trên cụm chậm hơn máy dev - đúng như khai báo, không phải cụm yếu.** Manifest ghi `limits.cpu: "1"`, mỗi pod chỉ được một CPU; máy dev nhiều nhân nên phục vụ song song tốt hơn. So sánh đúng phải là *một pod 1 CPU* với *một máy nhiều nhân*.
2. **Scale ngang tuyến tính - nay là số đo chứ không còn là suy luận kiến trúc.** Giữ nguyên tải 20 VU, chỉ tăng từ 1 lên 3 replica: thông lượng 3,63 → 10,83 hồ sơ/giây (**gấp 2,98 lần**), trung vị 5,49 s → 1,44 s (**nhanh 3,8 lần**). Đây là căn cứ thực nghiệm cho HorizontalPodAutoscaler ở `k8s/hpa.yaml`.

Dữ liệu đưa lên cụm là bản **đã ẩn danh**: họ tên, địa chỉ, điện thoại, số thẻ, mã bệnh nhân thay bằng ký tự giả **giữ nguyên độ dài** nên khối lượng phân tích không đổi. Hồ sơ bệnh nhân thật chỉ nằm trên máy dev, không đẩy lên cụm.

Riêng endpoint hỏi đáp AI bị rate-limit 10/phút/IP theo thiết kế (kiểm soát chi phí LLM) - muốn load test phần đó phải nâng ngưỡng tạm thời trên web admin; đây là tính năng, không phải giới hạn kỹ thuật.

6. Kết luận chịu tải

1. **Đường nóng nghiệp vụ ở mức mili-giây** (p95 173 ms, đo bằng k6) và không phụ thuộc AI - đáp ứng nhập liệu thời gian thực.
2. Ở 20 người dùng đồng thời trên **một máy dev**, hệ thống vẫn giữ **0% lỗi** và 15,57 hồ sơ/giây; độ trễ tăng tuyến tính theo tải chứ không sụp đổ.
3. **Trên cụm K8s thật**, một pod giới hạn 1 CPU cho p95 251 ms ở 1 VU; tăng lên 3 replica ở tải 20 VU cho thông lượng gấp **2,98 lần** - **scale ngang đã được chứng minh bằng số đo**, không còn là suy luận kiến trúc. Điểm cần theo dõi khi tải lớn là PostgreSQL (đã có PVC riêng, có thể nâng cấu hình/managed DB).
4. Rủi ro tải đã được chặn chủ động bằng rate-limit + giới hạn kích thước + streaming cho lô lớn.
5. Việc tiếp theo khi sản phẩm hóa: chạy bộ k6 ở mục 5 định kỳ trong CI/CD để có p95/p99 chính thức theo từng bản phát hành.